



Concordia University

Engineering and Computer Science

PROJECT REPORT

Name: Tudor Cosmin Suciu

ID No.: 40179863

Course: COMP 472

**I certify that this submission is my original work and
meets the Faculty's Expectations of Originality**

Signature: 

Date: 2024-11-26

1. Introduction

This report outlines the implementation and evaluation of various machine learning models for the CIFAR-10 image classification dataset. The models include Gaussian Naive Bayes (GNB), Decision Trees (DT), Multi-Layer Perceptrons (MLP), and Convolutional Neural Networks (CNN). Each model was evaluated on its ability to classify 10 object classes based on feature vectors extracted using a pre-trained ResNet-18, as well as on the CIFAR-10 images directly for the CNN. The metrics considered are accuracy, precision, recall, F1-score, and confusion matrices.

2. Additional details

GitHub repo: <https://github.com/cuscus555/COMP472-AI-Project.git>

3. Model Architectures and Training

Gaussian Naive Bayes (GNB)

- Custom GNB: Implemented using basic Python and NumPy libraries.
- Scikit GNB: Used Scikit-learn's GaussianNB implementation.
- Training Details: Feature vectors of size 50×1 extracted via ResNET-18 and PCA.

Decision Trees (DT)

- Custom DT: Implemented with maximum depths of 16, 50, and 100 using Python and NumPy.
- Variants: Analyzed how tree depth affects generalization and accuracy.
- Scikit DT: Used Scikit-learn's DecisionTreeClassifier.
- Training Details: Feature vectors of size 50×1 extracted via ResNET-18 and PCA.

Multi-Layer Perceptrons (MLP)

- Default MLP: A 3-layer network implemented as described in the project guidelines.
- Variants: Configurations with 2, 6 layers and varied hidden layer sizes (128 and 2048 units). Explored trade-offs between depth/size and performance.
- Training Details: Trained with SGD optimizer (momentum=0.9), cross-entropy loss and learning rate of 0.01. Number of epochs was kept at 10 for every model. Feature vectors of size 50×1 extracted via ResNET-18 and PCA were used.

Convolutional Neural Networks (CNN)

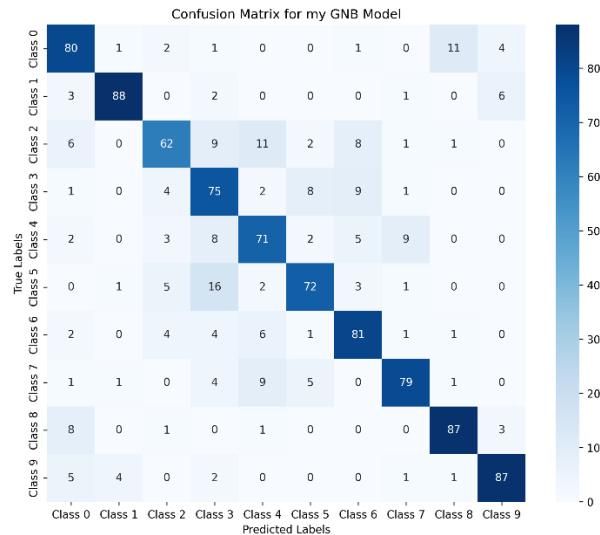
- Default CNN: VGG11 architecture using kernel size 3×3 , implemented as described in the project guidelines.
- Variants: Experimented with kernel size adjustments and additional layers.
- Training Details: Models were trained using cross-entropy loss, SGD optimizer (momentum=0.9), and learning rate of 0.01. Number of epochs used for the training was 45 since this was found to be a good training time to avoid overfitting or underfitting. The raw images from CIFAR-10 were used for training.

4. Evaluation and Results

Gaussian Naive Bayes (GNB)

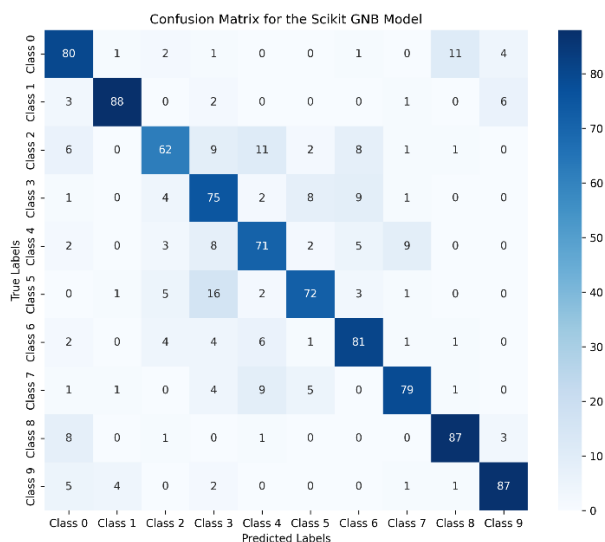
Custom GNB:

- Accuracy: 78.8%
- Observations: Strong recall for Class 1 and Class 8; frequent misclassification in Class 3 and 4.
- Challenges: Limited ability to capture complex patterns in the data.
- Confusion Matrix:



Scikit GNB:

- Accuracy: 78.8% (identical to Custom GNB).
- Observations: Performance mirrored the custom implementation, confirming consistency in modeling assumptions.
- Confusion Matrix:



Decision Trees (DT)

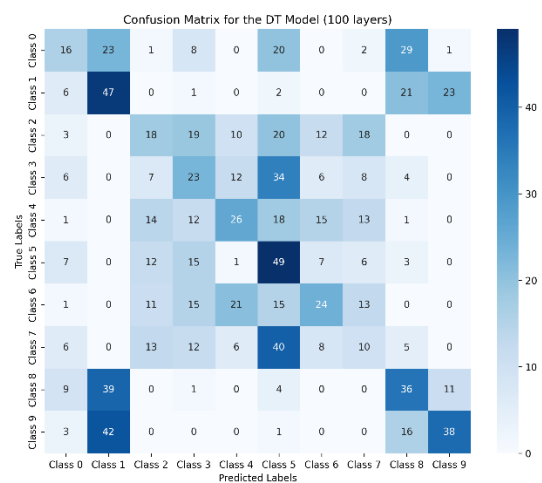
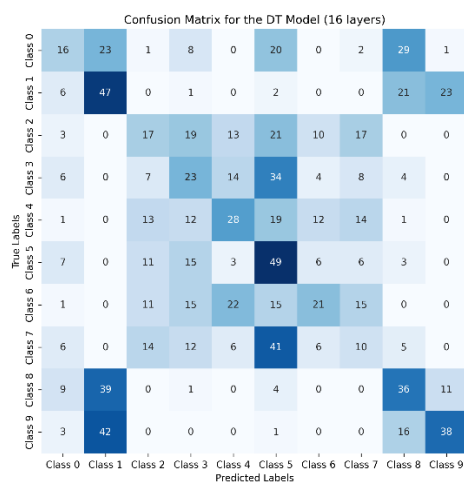
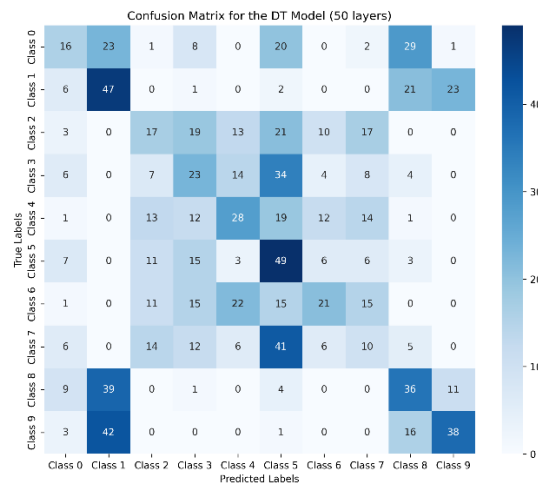
Custom DT (Depth Variations):

50 Layers: Accuracy of 28.5%.

16 Layers: Accuracy of 28.7%.

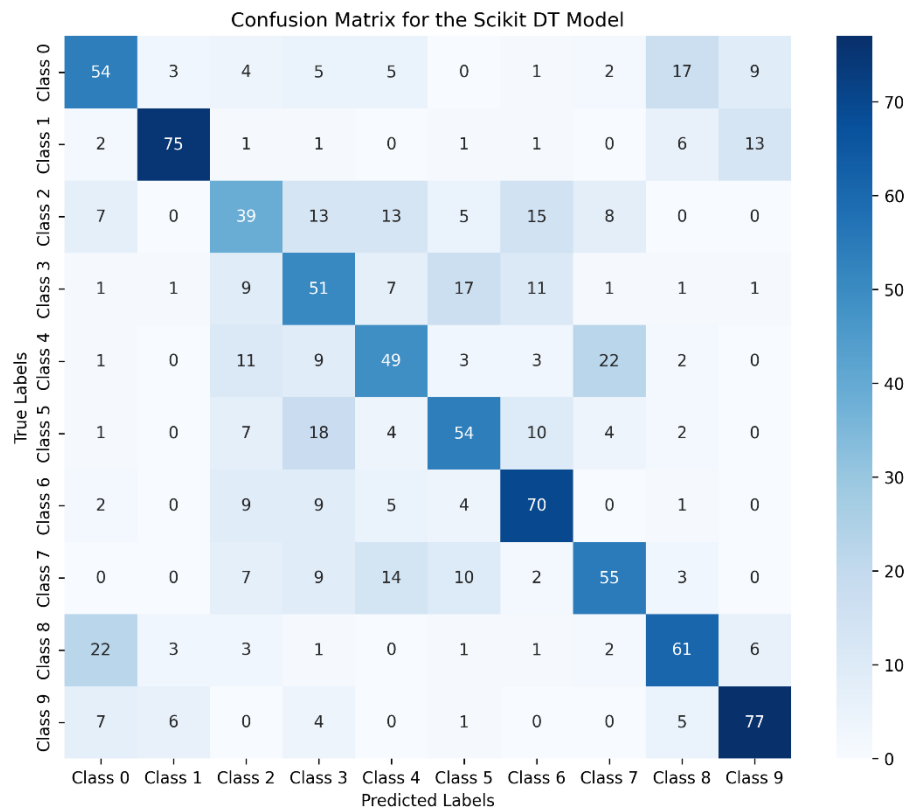
100 Layers: Accuracy of 28.5%.

- Accuracy: ~28.6%.
- Observations: Increasing depth showed no improvement, only longer training time, indicating overfitting on limited training data. Decreasing depth showed slightly better accuracy and faster to train, indicating the original 50 layers was too deep to begin with for the given dataset. From the confusion matrices we can conclude that the model has a tendency to misclassify classes 0 and 1 with classes 8 and 9 and vice versa, probably due to the similarities of the features of these classes.
- Confusion Matrices :



Scikit DT:

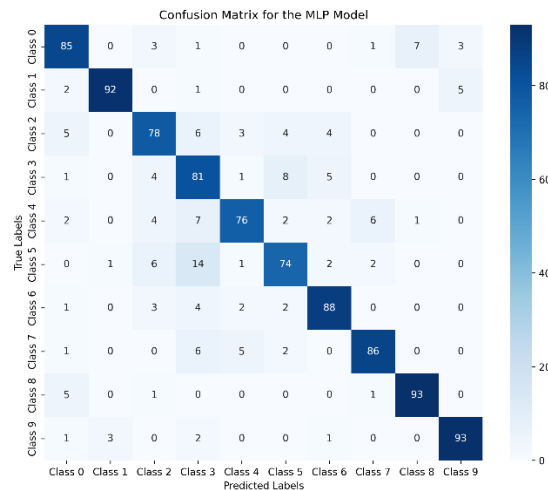
- Accuracy: 58.5%.
- Observations: Substantially better generalization compared to the custom implementation due to Scikit's optimized algorithms. But an accuracy rate of 58.5% is still low compared to the other models. From the confusion matrix we can conclude that it suffers from the same problem as my custom DT Model (to a lesser degree), probably due to the nature of the training data and the limitations of this model on that dataset.
- Confusion Matrix :



Multi-Layer Perceptrons (MLP)

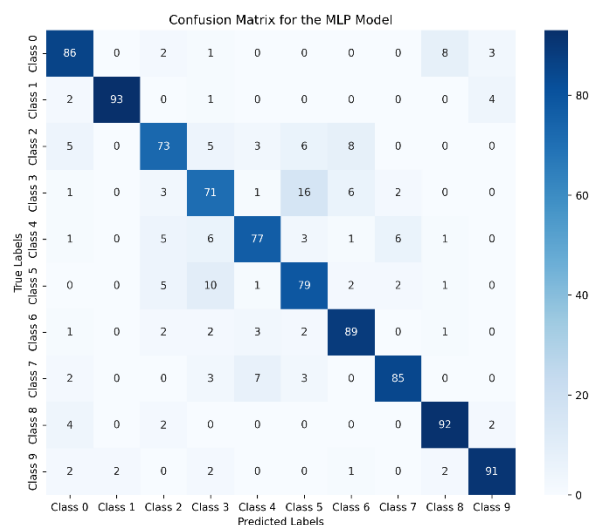
Default MLP (3 layers):

- Accuracy: 84.6%.
- Observations: Very high accuracy compared to the other models. High precision for Class 1 and Classes 9, 8 but struggles with Class 3 misclassification since it often thinks they are Class 5 or 6. It also tends to misclassify Class 5 objects as Class 3 objects. This is likely due to the feature resemblance of the aforementioned classes.
- Confusion Matrix :



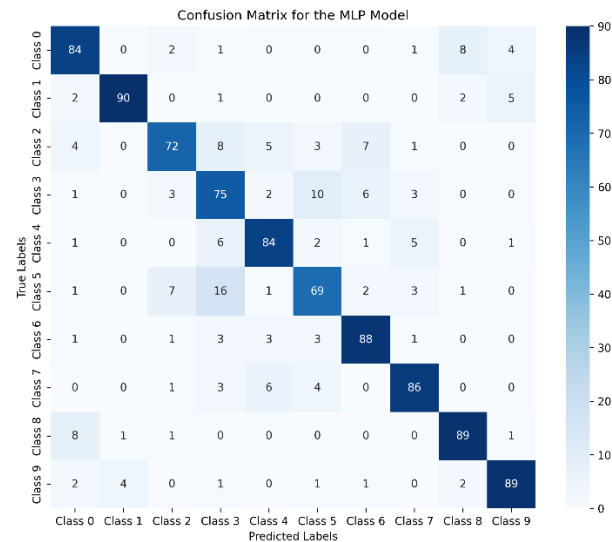
2-Layer MLP:

- Accuracy: 83.6%.
- Observations: Simplified architecture led to a drop in accuracy but the model is faster to train.
- Confusion Matrix :



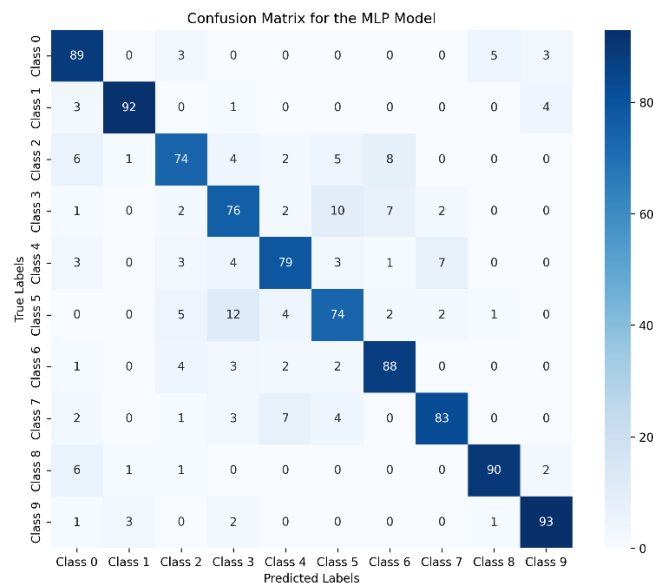
6-Layer MLP:

- Accuracy: 82.6%.
- Observations: Deeper architecture caused minor overfitting, reducing accuracy noticeably and model takes much longer to train.
- Confusion Matrix :
-



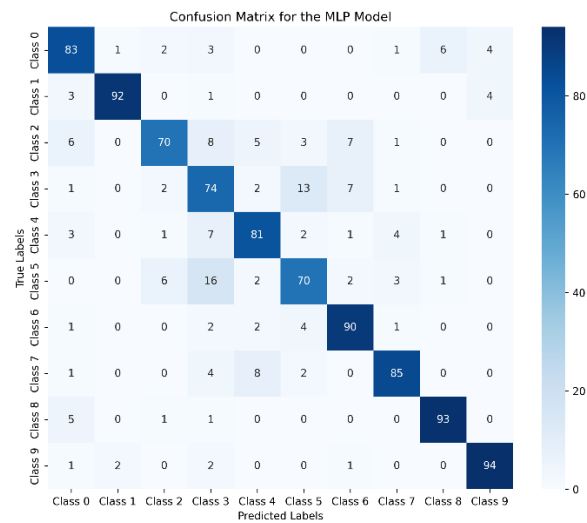
128 Hidden Layer Size MLP:

- Accuracy of 83.8%.
- Observations: Similar accuracy to default 3-layer model with 512 layer size and no noticeable change in training time.
- Confusion Matrix :



2048 Hidden Layer Size MLP:

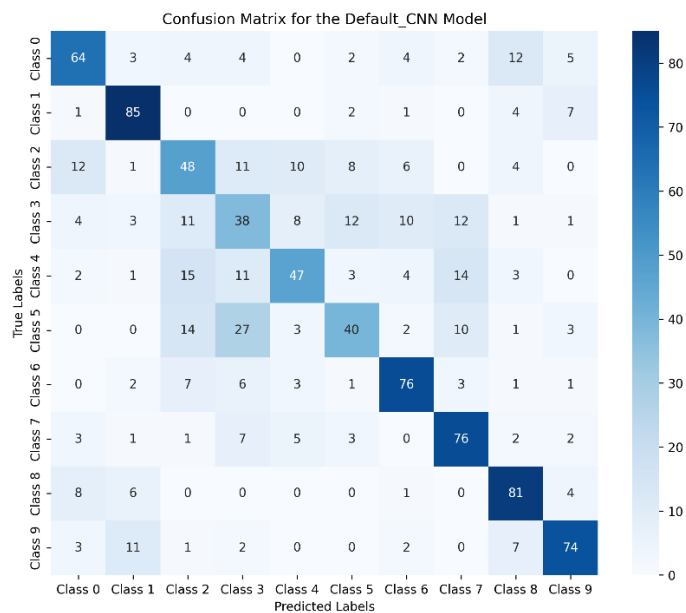
- Accuracy of 83.2%.
- Observations: Slightly worse accuracy and takes much longer to train. Larger layers increased computational cost without significant accuracy gain.
- Confusion Matrix :



Convolutional Neural Networks (CNN)

Default CNN (kernel size 3x3):

- Accuracy: 62.9%.
- Observations: Underperformed compared to MLP due to limited training data. Frequent Misclassifications of Classes 4 and 5
- Confusion Matrix :



Variants:

CNN with 8 conv. Layers added:

- Accuracy: 58.9%
- Observations: Very slow to train and accuracy drops significantly

CNN with 3 conv. Layers removed:

- Accuracy: 64.8%
- Observations: Faster to train and accuracy improves, meaning that the original model had too many layers for the given dataset.

CNN with 5x5 Kernel:

- Accuracy: 56.4%
- Observations: Much slower to train than 3x3 kernel, more computationally intensive and worse accuracy

CNN with 7x7 Kernel:

- Accuracy: 54.8%
- Observations: Very slow to train compared to 5x5 kernel, more computationally intensive and worse accuracy

Overall Observations: Larger kernels captured broader features but reduced granularity; smaller kernels were better for fine details. A 3x3 Kernel with less convolutional layers is ideal for this dataset.

5. Comparative Analysis

Best Model: Default MLP achieved the highest accuracy (84.6%), demonstrating its ability to balance complexity and generalization.

Naive Bayes: Effective for basic pattern recognition but lacked the capacity for complex feature discrimination.

Decision Trees: Limited by overfitting and sensitivity to depth.

CNN: Struggled with limited data; potential improvements with data augmentation.

6. Conclusion

The MLP models excelled in this task, showcasing their adaptability to feature-extracted datasets. While CNNs traditionally perform better with raw image data, the limited dataset size and small size of the images constrained their effectiveness. Decision Trees and Naive Bayes offered insights into model simplicity and generalization but fell short in accuracy.